RESUMELENS: FORMAL LANGUAGE-BASED RESUME SCREENING

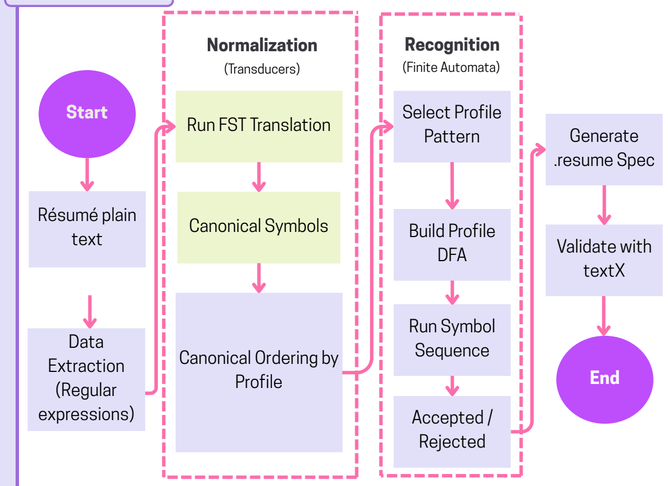
Maria Juliana Marin Shek Juan Pablo Gamboa Bryan Moreno Diaz

Universidad ICESI

INTRODUCTION

Recruitment screening must decide whether a résumé satisfies the qualifications of a professional profile, yet the same qualification appears under many names [JS, Javascript and JavaScript; sklearn, scikit learn and Scikit-learn]. ResumeLens treats screening as a formal-language problem: every stage of the pipeline is an explicit, analyzable formal model rather than an opaque heuristic. The system evaluates only whether the qualifications explicitly written in a résumé satisfy formally defined patterns, it does not rank candidates or make hiring decisions.

PIPELINE

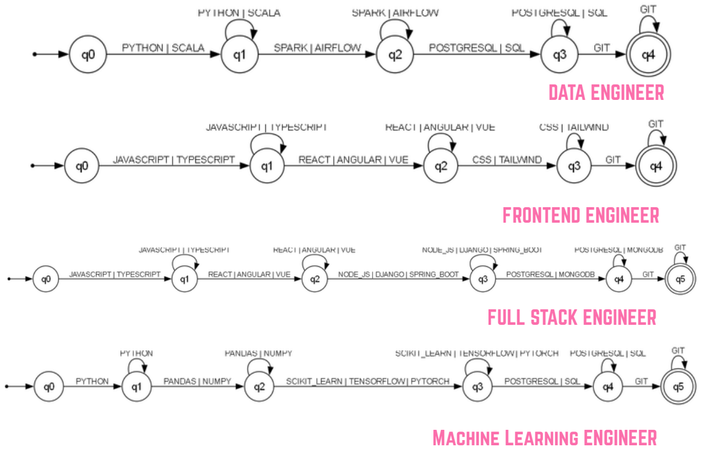


FORMALIZATION

Every transducer and automaton ships with its complete formal definition: the **7-tuple** for FSTs and the **5-tuple** for DFAs, with transition tables and Graphviz diagrams. The transition function δ is partial: a missing transition is an implicit trap state, so an unknown or out-of-order symbol kills the run and the sequence is rejected.

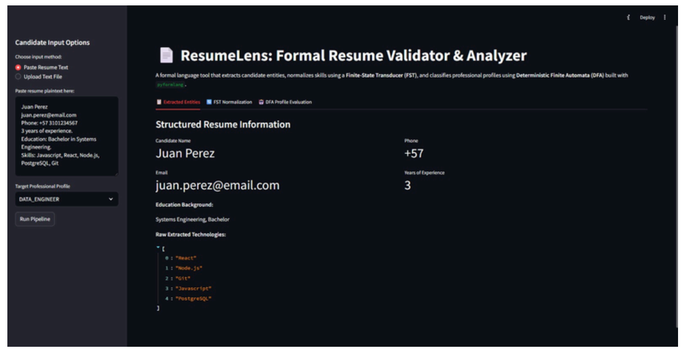
The DSL grammar is written in **EBNF,** terminals STRING, EMAIL, PHONE, DATE, CANON, … and non-terminals for candidate, contact, skills, experience, education and classification and implemented with **textX**, which derives the parser and the metamodel from a single grammar.

PROFILE AUTOMATON



$$M = (Q, \Sigma, \delta, q_0, F).$$

RESULTS



CONCLUSIONS

Formal language models give the screening pipeline properties heuristics cannot: each stage is independently specifiable, provable and testable. Limitations: a finite skill alphabet, exact-token extraction and binary accept/reject verdicts.

Future work: an interactive Streamlit web interface deployed to Community Cloud, a larger normalization lexicon, weighted and partial profile matching, and grammar simplification via normal forms.

PROFILES

Two profiles are predefined by the assignment; two are team-defined. Each profile is pure data : normalization rules, canonical order and automaton consumed by a single general pipeline.



VALIDATION

The DSL validator was exercised with a **30-specification suite:** 3 valid candidate profiles accepted and rendered to Markdown, and 25 invalid specifications correctly rejected. **5 lexical** (malformed dates, e-mails, lowercase skill tokens), **11 syntactic** (missing sections, broken lists, wrong section order) and **9 semantic** (duplicate skills or profiles, reversed periods, unknown canonical symbols).

The four profile DFAs classify **11 end-to-end cases** correctly, including both assignment reference résumés, alternative technology stacks (TypeScript / Angular / Django / MongoDB) and over-qualified candidates.

REFERENCES

J. E. Hopcroft, R. Motwani, J. D. Ullman. Introduction to Automata Theory, Languages, and Computation, 3rd ed. Pearson, 2007.

M. Mohri. "Finite-State Transducers in Language and Speech Processing." Computational Linguistics 23(2), 1997.

I. Dejanović, R. Vaderna, G. Milosavljević, Ž. Vuković. "TextX: A Python tool for Domain-Specific Languages implementation." Knowledge-Based Systems 115, 2017.

A. Romero. "Pyformlang: An Educational Library for Formal Language Manipulation." SIGCSE, 2021.

Code, formal definitions & test suite: github.com/brynzd/ResumeLens